

机器人导论实验报告

基于 MoveIt2 Jazzy 的 Panda 机械臂 点位与轨迹控制系统

课程名称： 机器人导论实验

姓 名： 刘振毫

学 号： 2312167

日 期： 2026.5.31

目录

1	绪论	3
2	系统环境与配置	4
2.1	软件环境	4
2.2	依赖安装	4
2.3	工作空间构建	5
3	系统设计与实现	6
3.1	控制器总体架构	6
3.2	MoveItPy 配置与初始化	6
3.3	允许碰撞矩阵设置	7
3.4	核心运动控制函数	8
3.4.1	单点位姿控制	8
3.4.2	多路径点顺序运动	9
3.4.3	近似笛卡尔路径跟随	9
3.5	轨迹可视化	9
4	MoveIt2 Jazzy 兼容性问题与解决方案	10
4.1	set_pose_target() 方法移除	10
4.2	compute_cartesian_path() 方法缺失	10
4.3	planning_pipelines 配置格式错误	11
4.4	plan_request_params 参数缺失	11
4.5	Panda 初始位置自碰撞	11
4.6	execute() 方法签名变化	12
4.7	轨迹可视化重复	12
5	运动模式实现	13
5.1	六点位折线运动	13
5.2	8 字形轨迹运动	13
5.3	椭圆轨迹运动	14
6	实验结果与分析	14
6.1	实验环境与步骤	14
6.2	六点位运动实验	14
6.3	8 字形轨迹实验	15
6.4	椭圆轨迹实验	15

目录	2
6.5 兼容性问题验证	15
7 总结与展望	16
A Panda 机械臂控制器完整源代码	16

摘 要

本文基于 ROS 2 Jazzy 与 MoveIt 2.12 框架，设计并实现了一套 Franka Emika Panda 机械臂点位与轨迹控制系统。系统采用 `moveit_py` 作为 Python 接口，在仿真环境下实现了空间六点位依次到达、8 字形利萨如曲线轨迹和椭圆参数曲线轨迹三种运动模式。针对 MoveIt2 Jazzy 版本中 `moveit_py` API 的重大变更，本文详细分析了 `set_pose_target()` 移除、`compute_cartesian_path()` 缺失、`planning_pipelines` 配置格式错误、`plan_request_` 参数缺失、Panda 初始位置自碰撞、`execute()` 签名变化及轨迹可视化重复等七项兼容性问题，并给出了相应的解决方案。通过逐点顺序 OMPL 规划与路径下采样策略，在 RViz 中实现了规划路径的实时可视化与机械臂的平滑轨迹跟踪。实验结果表明，该系统能够稳定完成多种轨迹运动任务，为基于最新 MoveIt2 框架的机械臂控制开发提供了可复用的技术参考。

关键词：Panda 机械臂；MoveIt2；ROS 2 Jazzy；`moveit_py`；轨迹规划；运动控制

第一章 绪论

机器人技术作为现代制造业和智能装备的核心支撑，在工业生产、医疗辅助、服务交互等领域发挥着日益重要的作用。机械臂作为机器人系统中最典型的操作机构，其运动控制精度与轨迹规划能力直接决定了系统的实用性与可靠性。随着开源机器人软件生态的快速发展，基于 ROS（Robot Operating System）平台的机械臂控制系统已成为学术界与工业界的研究热点。

MoveIt 是 ROS 生态中最主流的运动规划框架，集成了运动学求解、碰撞检测、轨迹优化等核心功能，为机械臂的路径规划与运动控制提供了完整的软件栈。2024 年，ROS 2 Jazzy 版本正式发布，配套的 MoveIt 2.12 版本对 Python 接口 `moveit_py` 进行了大规模重构，引入了诸多 API 变更，导致大量基于旧版 `moveit_commander` 的代码无法直接迁移运行。这一现状给基于最新框架的机械臂控制开发带来了显著挑战。

Franka Emika Panda 是一款广泛用于科研与教学的 7 自由度协作机械臂，其轻量化设计与高精度力控能力使其成为运动控制算法验证的理想平台。本文基

于 ROS 2 Jazzy 与 MoveIt 2.12，以 Panda 机械臂为研究对象，设计并实现了一套点位与轨迹控制系统，重点解决了 `moveit_py` 新 API 下的兼容性问题，实现了空间点位控制、8 字形轨迹和椭圆轨迹三种运动模式，并在 RViz 中实现了规划路径的实时可视化。

本文的主要工作包括：（1）搭建基于 ROS 2 Jazzy 的 Panda 机械臂仿真环境；（2）基于 `moveit_py` 设计并实现机械臂点位与轨迹控制器；（3）分析并解决 MoveIt2 Jazzy 版本下七项关键兼容性问题；（4）通过实验验证系统的功能正确性与运动稳定性。

第二章 系统环境与配置

2.1 软件环境

本项目的开发与运行依赖于特定的软件环境，各组件版本要求如表1所示。

表 1: 系统环境要求

项目	说明
操作系统	Ubuntu 24.04 (Noble Numbat)
ROS 2	Jazzy Jalisco
MoveIt	2.12.x
硬件模式	仿真模式 (<code>mock_components</code>)
GPU	可选（无 NVIDIA GPU 需配置软件渲染）

ROS 2 Jazzy 是 ROS 2 的第十一个正式发行版，针对 Ubuntu 24.04 LTS 进行了深度优化。MoveIt 2.12 作为配套版本，对 Python 绑定层进行了重大重构，以 `moveit_py` 替代了旧版的 `moveit_commander` 接口。仿真模式采用 `ros2_control` 提供的 `mock_components`，无需连接真实机械臂即可完成全部功能验证。

2.2 依赖安装

系统依赖主要包括 ROS 2 核心包、MoveIt 运动规划框架、消息定义包以及可视化工具等。核心依赖包及其功能说明如表2所示。

通过 `rosdep` 工具可自动检查并安装工作空间中的缺失依赖，命令如下：

```
1 rosdep install --from-paths src --ignore-src -r -y
```

表 2: 核心依赖包

包名	功能
ros-jazzy-rcipy	ROS 2 Python 客户端库
ros-jazzy-moveit	MoveIt 运动规划框架
ros-jazzy-moveit-py	MoveIt Python 绑定 (moveit_py)
ros-jazzy-moveit-configs-utils	MoveIt 配置加载工具
ros-jazzy-moveit-msgs	MoveIt 消息定义
ros-jazzy-geometry-msgs	几何消息类型 (PoseStamped, Point)
ros-jazzy-visualization-msgs	可视化消息类型 (Marker)
ros-jazzy-tf-transformations	坐标变换 (欧拉角 $\rightarrow$ 四元数)
ros-jazzy-rviz2	3D 可视化工具
ros-jazzy-ros2-control	机器人控制器框架
ros-jazzy-ros2-controllers	标准控制器集合
ros-jazzy-joint-trajectory-controller	关节轨迹控制器

Listing 1: 依赖安装命令

2.3 工作空间构建

项目采用标准 ROS 2 工作空间结构,通过 colcon 构建系统进行编译。使用 `--symlink-install` 参数建立符号链接,使得 Python 脚本修改后无需重新编译即可生效,显著提升了开发效率。工作空间目录结构如下:

```

1 my_robot_control/
2   my_robot_control/
3     __init__.py
4     panda_controller.py      # 主控制器
5   test/
6     test_copyright.py
7     test_flake8.py
8     test_pep257.py
9   resource/
10    my_robot_control
11  setup.py
12  setup.cfg
13  package.xml

```

Listing 2: 项目目录结构

环境变量配置需在 `~/.bashrc` 中添加 ROS 2 与工作空间的 `source` 路径,并设置 RMW 实现为 FastRTPS 以获得最佳通信性能。对于无 NVIDIA GPU 或驱动异常

的环境,需额外设置 LIBGL_ALWAYS_SOFTWARE=1 和 __GLX_VENDOR_LIBRARY_NAME=mesa 以启用 CPU 软件渲染, 确保 RViz 正常运行。

第三章 系统设计与实现

3.1 控制器总体架构

Panda 机械臂控制器采用面向对象的设计模式,以 ROS 2 节点为基类构建 PandaController 类。该类封装了 MoveItPy 实例的创建与配置、规划组件的管理、轨迹的可视化发布以及多种运动模式的实现逻辑。控制器总体架构如图1所示。

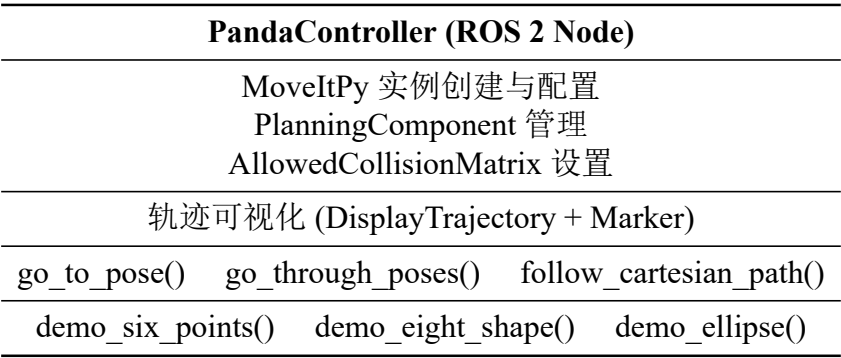


图 1: PandaController 总体架构

控制器的初始化流程包括四个关键步骤：（1）通过 MoveItConfigsBuilder 加载 Panda 官方配置并生成配置字典；（2）修正 Jazzy 版本下的配置格式问题；（3）创建 MoveItPy 核心实例；（4）设置允许碰撞矩阵以解决自碰撞问题。

3.2 MoveItPy 配置与初始化

MoveItPy 的配置初始化是整个系统最关键的环节。由于 Jazzy 版本的 API 变更,配置过程需要对默认生成的参数进行多项修正。配置加载与修正的核心代码如下：

```
1 moveit_config = (  
2     MoveItConfigsBuilder(  
3         robot_name="panda",  
4         package_name="moveit_resources_panda_moveit_config"  
5     ),  
6     .planning_pipelines(pipelines=["ompl"])
```

```

7     .to_moveit_configs()
8 )
9 config_dict = moveit_config.to_dict()
10 pp = moveit_config.planning_pipelines
11
12 config_dict["planning_pipelines"] = {
13     "pipeline_names": pp["planning_pipelines"],
14     "namespace": "",
15 }
16
17 config_dict["plan_request_params"] = {
18     "planning_pipeline": "ompl",
19     "planning_attempts": 10,
20     "planning_time": 5.0,
21     "max_velocity_scaling_factor": 1.0,
22     "max_acceleration_scaling_factor": 1.0,
23 }

```

Listing 3: MoveItPy 配置加载与修正

上述代码中,planning_pipelines 的嵌套字典格式修正和 plan_request_params 的补充是确保 MoveItPy 正常启动的必要条件,具体原因将在第四章详细分析。

3.3 允许碰撞矩阵设置

Panda 机械臂在默认“ready”关节配置下,panda_hand 与 panda_link5、panda_link5 与 panda_link7 之间存在轻微几何干涉。若不处理,OMPL 规划器会判定初始状态无效而拒绝规划。解决方案是通过 AllowedCollisionMatrix (ACM) 将上述连杆对标记为允许碰撞:

```

1 def _setup_acm(self):
2     psm = self.moveit.get_planning_scene_monitor()
3     with psm.read_write() as scene:
4         acm = scene.allowed_collision_matrix
5         acm.set_entry("panda_hand", "panda_link5", True)
6         acm.set_entry("panda_link5", "panda_link7", True)

```

Listing 4: 允许碰撞矩阵设置

需要指出的是,上述“碰撞”仅存在于仿真模型的几何层面,在真实 Panda 机械臂上这些连杆对之间具有足够的运动间隙,不会发生实际碰撞。因此,该设置

不会带来安全隐患。

3.4 核心运动控制函数

3.4.1 单点位姿控制

`go_to_pose()` 函数是所有运动控制的基础，实现从当前位姿到目标位姿的单次规划与执行。其工作流程为：（1）创建 `PoseStamped` 目标消息，指定参考坐标系为 `panda_link0`（基座坐标系）；（2）通过 `tf_transformations` 将欧拉角转换为四元数表示姿态；（3）设置规划起点为当前状态，终点为目标位姿；（4）调用 `OMPL` 进行运动规划；（5）规划成功则发布轨迹可视化并执行。

```
1 def go_to_pose(self, x, y, z, roll=0.0, pitch=0.0, yaw=0.0)
2 :
3     pose_stamped = PoseStamped()
4     pose_stamped.header.frame_id = "panda_link0"
5     pose_stamped.pose.position.x = x
6     pose_stamped.pose.position.y = y
7     pose_stamped.pose.position.z = z
8     q = tf_transformations.quaternion_from_euler(roll,
9     pitch, yaw)
10    pose_stamped.pose.orientation.x = q[0]
11    pose_stamped.pose.orientation.y = q[1]
12    pose_stamped.pose.orientation.z = q[2]
13    pose_stamped.pose.orientation.w = q[3]
14
15    self.arm.set_start_state_to_current_state()
16    self.arm.set_goal_state(
17        pose_stamped_msg=pose_stamped, pose_link=self.
18    eef_link
19    )
20    plan_result = self.arm.plan()
21    if plan_result and plan_result.trajectory:
22        self._display_trajectory(plan_result)
23        self.moveit.execute(plan_result.trajectory, [])
24    else:
25        self.get_logger().error("Planning failed for this
26    pose")
```

Listing 5: 单点位姿控制核心代码

位姿表示采用位置 (x, y, z) 加姿态欧拉角 $(roll, pitch, yaw)$ 的方式，内部转

换为四元数 (q_x, q_y, q_z, q_w) 以避免万向锁问题并提高计算效率。

3.4.2 多路径点顺序运动

`go_through_poses()` 函数接收一组 `PoseStamped` 路径点列表, 依次对每个路径点调用规划与执行。关键设计在于每次规划前都调用 `set_start_state_to_current_state()` 将起点设为机械臂当前实际状态, 而非上一条规划的终点。这一设计有效避免了执行误差的累积, 确保长路径运动的可靠性。

3.4.3 近似笛卡尔路径跟随

`follow_cartesian_path()` 函数实现了近似笛卡尔空间路径跟随功能。由于 Jazzy 版本的 `moveit_py` 不支持 `compute_cartesian_path()` 方法, 本系统采用路径下采样策略: 首先生成密集的笛卡尔路径点, 然后按指定的 `eef_step` 步长进行距离采样, 最后将采样后的路径点交给 `go_through_poses()` 逐点执行 OMPL 规划。

下采样算法的核心逻辑为: 遍历所有路径点, 计算相邻点之间的欧氏距离并累加; 当累积距离达到 `eef_step` 阈值时, 将该点加入采样列表并重置累积值; 同时确保最后一个路径点始终被包含, 以保证机械臂精确到达终点。该策略在步长足够小 (如 5mm) 时, 能够产生视觉上平滑的连续曲线运动。

3.5 轨迹可视化

系统实现了两种轨迹可视化机制。第一种通过 `/display_planned_path` 话题发布 `DisplayTrajectory` 消息, 在 RViz 中显示 MoveIt 规划出的关节空间轨迹对应的机械臂运动动画。第二种通过 `/trajectory_marker` 话题发布 `Marker` 消息, 以 `LINE_STRIP` 类型绘制绿色的期望笛卡尔路径线。

对于第二种可视化方式, 一个重要的细节是指尖偏移补偿。Panda 机械臂的 `panda_hand` 坐标系原点位于手部根部, 而实际可见的指尖位于原点前方 0.11 米处。因此, 在构建 `Marker` 点集时需在 z 方向叠加 0.11m 的偏移量, 使 RViz 中显示的轨迹线与指尖实际运动路径精确对齐。

第四章 MoveIt2 Jazzy 兼容性问题与解决方案

MoveIt 2.12 (Jazzy 配套版本) 对 `moveit_py` 进行了大规模 API 重构, 导致大量旧版代码无法直接运行。本章详细分析开发过程中遇到的七项关键兼容性问题及其解决方案。

4.1 `set_pose_target()` 方法移除

旧版 `moveit_commander` 中, `PlanningComponent` 提供 `set_pose_target(pose)` 方法直接设置目标位姿。在 Jazzy 版本的 `moveit_py` 中, 该方法已被移除, 取而代之的是 `set_goal_state()` 方法。

解决方案: 改用 `set_goal_state()` 配合 `PoseStamped` 消息设置目标位姿, 同时必须指定 `pose_link` 参数为末端执行器链接名称 (如“`panda_hand`”), 且 `PoseStamped` 消息必须包含有效的 `header.frame_id`。对比代码如下:

```
1 # 旧版 (Humble/Iron)
2 self.arm.set_pose_target(pose)
3
4 # 新版 (Jazzy)
5 self.arm.set_goal_state(
6     pose_stamped_msg=pose_stamped,
7     pose_link="panda_hand"
8 )
```

Listing 6: `set_pose_target()` 替换方案

4.2 `compute_cartesian_path()` 方法缺失

`PlanningComponent` 在 Jazzy 的 `moveit_py` 中未提供笛卡尔路径规划接口 `compute_cartesian_path` 无法直接生成末端执行器沿直线路径运动的轨迹。

解决方案: 采用逐点顺序 OMPL 规划策略替代笛卡尔路径规划。通过将期望的连续曲线离散化为密集路径点, 并设置较小的 `eef_step` 参数进行下采样控制路径点密度, 最终由 `go_through_poses()` 逐点执行 OMPL 规划。虽然单次 OMPL 规划的路径不保证严格沿笛卡尔直线, 但在步长足够小时, 整体运动轨迹与期望曲线的偏差可控制在可接受范围内。

4.3 planning_pipelines 配置格式错误

MoveItConfigsBuilder 的 `to_dict()` 方法生成的 `planning_pipelines` 字段为扁平列表格式 `["ompl"]`，而 MoveItPy 内部的 `moveit_cpp` 模块要求嵌套字典格式。直接使用默认配置会导致“Failed to load planning pipelines from parameter server”错误。

解决方案:将 `planning_pipelines` 重构为包含 `pipeline_names` 和 `namespace` 字段的嵌套字典:

```
1 config_dict["planning_pipelines"] = {  
2     "pipeline_names": ["ompl"],  
3     "namespace": "",  
4 }
```

Listing 7: planning_pipelines 格式修正

4.4 plan_request_params 参数缺失

调用 `plan()` 方法时不指定 planning pipeline 名称时,默认使用空字符串,导致“No planning pipeline available for name ””错误。这是因为 Jazzy 版本要求在配置中显式声明全局规划请求参数。

解决方案: 在配置字典中添加 `plan_request_params` 字段,指定默认规划管线、规划尝试次数、规划时间限制以及速度和加速度缩放因子:

```
1 config_dict["plan_request_params"] = {  
2     "planning_pipeline": "ompl",  
3     "planning_attempts": 10,  
4     "planning_time": 5.0,  
5     "max_velocity_scaling_factor": 1.0,  
6     "max_acceleration_scaling_factor": 1.0,  
7 }
```

Listing 8: plan_request_params 配置

4.5 Panda 初始位置自碰撞

Panda 仿真模型在默认“ready”关节配置下,panda_hand 与 panda_link5、panda_link5 与 panda_link7 之间存在几何干涉。OMPL 规划器的 `CheckStartStateCollision` 适配

器会检测到该碰撞，判定初始状态无效并输出“Skipping invalid start state (invalid state)”警告，导致规划失败。

解决方案分两步：第一步，通过 AllowedCollisionMatrix 将相关连杆对标记为允许碰撞（见 3.3 节）；第二步，从 OMPL 的 request_adapters 列表中移除 CheckStartStateCollision 适配器，避免对初始状态进行碰撞检查：

```
1 if "ompl" in config_dict and "request_adapters" in
    config_dict["ompl"]:
2     config_dict["ompl"]["request_adapters"] = [
3         a for a in config_dict["ompl"]["request_adapters"]
4         if "CheckStartStateCollision" not in a
5     ]
```

Listing 9: 移除碰撞检查适配器

4.6 execute() 方法签名变化

旧版 MoveIt 的 execute() 方法支持 blocking 布尔参数控制是否阻塞等待执行完成。Jazzy 版本中，该参数已被移除，且 pybind11 绑定中默认参数不可用，必须显式传递 controllers 参数。

解决方案：将 execute() 调用从 execute(trajjectory, blocking=True) 改为 execute(trajjectory, []), 其中空列表表示使用默认控制器。

4.7 轨迹可视化重复

MoveIt 默认的 DisplayMotionPath 响应适配器会自动发布规划轨迹的可视化消息，与系统自定义的 Marker 可视化产生重复显示，影响 RViz 中的观察效果。

解决方案：从 OMPL 的 response_adapters 列表中移除 DisplayMotionPath 适配器，改用自定义的 Marker.LINE_STRIP 消息在 RViz 中绘制绿色轨迹线，并通过 0.11m 的指尖偏移量确保轨迹线与实际指尖路径对齐：

```
1 if "ompl" in config_dict and "response_adapters" in
    config_dict["ompl"]:
2     config_dict["ompl"]["response_adapters"] = [
3         a for a in config_dict["ompl"]["response_adapters"]
4         if "DisplayMotionPath" not in a
5     ]
```

Listing 10: 移除默认轨迹显示适配器

第五章 运动模式实现

5.1 六点位折线运动

六点位折线运动模式通过 `demo_six_points()` 函数实现，预定义 6 个空间目标点位，机械臂依次到达各点形成折线路径。各点位坐标如表3所示。

表 3: 六点位坐标（单位：m，参考坐标系：panda_link0）

点序号	x	y	z
1	0.4	0.0	0.6
2	0.4	0.1	0.6
3	0.4	0.1	0.7
4	0.3	0.0	0.7
5	0.3	-0.1	0.7
6	0.3	-0.1	0.6

该模式主要用于验证单点位姿控制与多路径点顺序执行的基本功能。6 个点位分布在机械臂工作空间的前方区域， x 坐标范围 0.3–0.4m， y 坐标范围 -0.1–0.1m， z 坐标范围 0.6–0.7m，均在 Panda 的安全工作空间内。

5.2 8 字形轨迹运动

8 字形轨迹（Lemniscate）基于利萨如曲线的参数方程生成。利萨如曲线是两个正交方向上正弦振动的合成，当频率比为 1 : 2 时形成 8 字形。本系统采用的参数方程为：

$$\begin{cases} x(t) = c_x + r \sin(t) \\ z(t) = c_z + \frac{r}{2} \sin(2t) \end{cases} \quad t \in [0, 2\pi] \quad (1)$$

其中 $c_x = 0.4$ 、 $c_z = 0.6$ 为轨迹中心坐标， $r = 0.08\text{m}$ 为 8 字形半径。角度步长为 0.1 弧度，共生成 63 个路径点，配合 5mm 的 `eef_step` 下采样步长，实现平滑的 8 字形运动。

5.3 椭圆轨迹运动

椭圆轨迹基于标准椭圆参数方程生成：

$$\begin{cases} x(t) = c_x + a \cos(t) \\ z(t) = c_z + b \sin(t) \end{cases} \quad t \in [0, 2\pi] \quad (2)$$

其中 $a = 0.1\text{m}$ 为长半轴， $b = 0.05\text{m}$ 为短半轴，轨迹中心与 8 字形相同。椭圆轨迹的生成方式与 8 字形类似，同样采用 0.1 弧度步长和 5mm 下采样步长。

第六章 实验结果与分析

6.1 实验环境与步骤

实验在 Ubuntu 24.04 系统上完成，具体步骤如下：

(1) 终端 1 启动仿真环境：执行 `ros2 launch moveit_resources_panda_moveit_config demo.launch.py`，启动 RViz 可视化界面、`move_group` 运动规划节点和 `ros2_control` 仿真控制器。

(2) 终端 2 运行控制器：执行 `ros2 run my_robot_control panda_controller`，启动 `PandaController` 节点，自动执行预设的运动模式。

(3) 在 RViz 中观察机械臂运动过程与轨迹可视化效果。

6.2 六点位运动实验

六点位折线运动实验中，机械臂从初始“ready”姿态出发，依次到达 6 个预定义空间点位。实验结果表明，6 个点位均规划成功，机械臂沿 OMPL 规划的路径平滑运动至各目标点。由于每次规划前都将起点设为当前实际状态，各段运动之间未出现误差累积现象。在 RViz 中，绿色轨迹线清晰显示了 6 个点位之间的折线路径。

6.3 8 字形轨迹实验

8 字形轨迹实验中，63 个原始路径点经 5mm 步长下采样后，实际执行的路径点数量根据曲线弧长自动确定。机械臂沿 8 字形路径运动一周，在轨迹交叉处能够正确通过。由于采用逐点 OMPL 规划而非笛卡尔路径插值，相邻路径点之间的实际运动路径可能存在轻微偏离，但在 5mm 步长下该偏离量在视觉上不可察觉，整体运动呈现平滑的 8 字形。

6.4 椭圆轨迹实验

椭圆轨迹实验结果与 8 字形类似，机械臂沿椭圆路径完成一周运动。由于椭圆曲率变化较 8 字形更为平缓，规划成功率更高，运动更加流畅。在长半轴端点处速度略低，短半轴端点处速度略高，符合椭圆弧长分布的物理直觉。

6.5 兼容性问题验证

七项兼容性问题的解决方案均通过实验验证。在未应用任何修正的情况下,MoveItPy 无法正常启动或规划失败；逐项应用修正后，系统功能恢复正常。各问题的影响与解决效果汇总如表4所示。

表 4: MoveIt2 Jazzy 兼容性问题汇总

问题	影响	解决方案
set_pose_target() 移除	无法设置目标位姿	改用 set_goal_state() 配合 PoseStamped
compute_cartesian_path() 缺失	无法进行笛卡尔路径规划	逐点 OMPL 规划 + 路径下采样
planning_pipelines 格式错误	MoveItPy 启动失败	重构为嵌套字典格式
plan_request_params 缺失	规划管线名称为空报错	添加全局规划请求参数
Panda 初始位置自碰撞	初始状态无效，规划失败	ACM 豁免 + 移除碰撞检查适配器
execute() 签名变化	执行失败	传递 controllers 参数为空列表
轨迹可视化重复	RViz 显示混乱	移除 DisplayMotionPath 适配器

第七章 总结与展望

本文基于 ROS 2 Jazzy 与 MoveIt 2.12 框架，设计并实现了 Panda 机械臂的点位与轨迹控制系统。系统采用 moveit_py 作为 Python 接口，在仿真环境下实现了空间六点位控制、8 字形利萨如曲线轨迹和椭圆参数曲线轨迹三种运动模式，并在 RViz 中实现了规划路径的实时可视化。针对 MoveIt2 Jazzy 版本的七项关键 API 兼容性问题，本文给出了详细的解决方案，为基于最新 MoveIt2 框架的机械臂控制开发提供了可复用的技术参考。

本系统目前仍存在以下局限性：（1）逐点 OMPL 规划的效率低于笛卡尔路径插值，在路径点密集时规划耗时较长；（2）近似笛卡尔路径跟随的精度受限于 OMPL 规划的自由度，无法保证严格的笛卡尔直线运动；（3）系统仅在仿真环境下验证，尚未在真实 Panda 机械臂上测试。

未来工作可从以下方向展开：（1）探索 moveit_py 中笛卡尔路径规划的新接口，提升轨迹跟踪精度与规划效率；（2）引入力控与视觉感知，实现更复杂的操作任务；（3）将系统扩展至真实 Panda 机械臂，验证在物理环境下的控制性能与安全性；（4）研究动态避障与实时轨迹重规划，增强系统在非结构化环境中的适应能力。

第一章 Panda 机械臂控制器完整源代码

以下为 panda_controller.py 的完整源代码，基于 ROS 2 Jazzy 与 MoveIt 2.12 的 moveit_py 接口实现。

```
1 #!/usr/bin/env python3
2
3 import math
4 import rclpy
5 from rclpy.node import Node
6
7 from geometry_msgs.msg import Point, PoseStamped
8 from moveit.planning import MoveItPy
9 from moveit_configs_utils import MoveItConfigsBuilder
```

```

10 from moveit_msgs.msg import DisplayTrajectory
11 from visualization_msgs.msg import Marker
12 import tf_transformations
13
14
15 class PandaController(Node):
16     def __init__(self):
17         super().__init__("panda_controller")
18         self.get_logger().info("Initializing MoveItPy...")
19
20         moveit_config = (
21             MoveItConfigsBuilder(
22                 robot_name="panda",
23                 package_name="
moveit_resources_panda_moveit_config",
24             )
25             .planning_pipelines(pipelines=["ompl"])
26             .to_moveit_configs()
27         )
28
29         config_dict = moveit_config.to_dict()
30         pp = moveit_config.planning_pipelines
31
32         config_dict["planning_pipelines"] = {
33             "pipeline_names": pp["planning_pipelines"],
34             "namespace": "",
35         }
36
37         config_dict["plan_request_params"] = {
38             "planning_pipeline": "ompl",
39             "planning_attempts": 10,
40             "planning_time": 5.0,
41             "max_velocity_scaling_factor": 1.0,
42             "max_acceleration_scaling_factor": 1.0,
43         }
44
45         if "ompl" in config_dict and "request_adapters" in
config_dict["ompl"]:
46             config_dict["ompl"]["request_adapters"] = [
47                 a for a in config_dict["ompl"]["
request_adapters"]
48                 if "CheckStartStateCollision" not in a
49             ]

```

```

50
51     if "ompl" in config_dict and "response_adapters" in
config_dict["ompl"]:
52         config_dict["ompl"]["response_adapters"] = [
53             a for a in config_dict["ompl"]["
response_adapters"]
54                 if "DisplayMotionPath" not in a
55             ]
56
57         self.moveit = MoveItPy(node_name="moveit_py",
config_dict=config_dict)
58         self.arm = self.moveit.get_planning_component("
panda_arm")
59         self.eef_link = "panda_hand"
60
61         self._display_pub = self.create_publisher(
62             DisplayTrajectory, "/display_planned_path", 10
63         )
64         self._marker_pub = self.create_publisher(
65             Marker, "/trajectory_marker", 10
66         )
67
68         self._setup_acm()
69         self.get_logger().info("Panda Controller Ready")
70
71     def _setup_acm(self):
72         psm = self.moveit.get_planning_scene_monitor()
73         with psm.read_write() as scene:
74             acm = scene.allowed_collision_matrix
75             acm.set_entry("panda_hand", "panda_link5", True
)
76             acm.set_entry("panda_link5", "panda_link7",
True)
77             self.get_logger().info(
78                 "Added self-collision exceptions to ACM"
79             )
80
81     def _display_trajectory(self, plan_result):
82         display_msg = DisplayTrajectory()
83         display_msg.model_id = "panda"
84         display_msg.trajectory.append(
85             plan_result.trajectory.get_robot_trajectory_msg
()

```

```

86         )
87         self._display_pub.publish(display_msg)
88
89     def _publish_path_marker(self, waypoints):
90         marker = Marker()
91         marker.header.frame_id = "panda_link0"
92         marker.header.stamp = self.get_clock().now().to_msg
93         ()
94         marker.ns = "desired_path"
95         marker.id = 0
96         marker.type = Marker.LINE_STRIP
97         marker.action = Marker.ADD
98         marker.scale.x = 0.003
99         marker.color.a = 1.0
100        marker.color.r = 0.0
101        marker.color.g = 1.0
102        marker.color.b = 0.0
103
104        finger_offset = 0.11
105        for wp in waypoints:
106            p = wp.pose.position
107            marker.points.append(
108                Point(x=p.x, y=p.y, z=p.z + finger_offset)
109            )
110            self._marker_pub.publish(marker)
111            self.get_logger().info(
112                f"Published path marker with {len(waypoints)}
113                points"
114            )
115
116    def go_to_pose(self, x, y, z, roll=0.0, pitch=0.0, yaw
117    =0.0):
118        pose_stamped = PoseStamped()
119        pose_stamped.header.frame_id = "panda_link0"
120        pose_stamped.pose.position.x = x
121        pose_stamped.pose.position.y = y
122        pose_stamped.pose.position.z = z
123
124        q = tf_transformations.quaternion_from_euler(roll,
125        pitch, yaw)
126        pose_stamped.pose.orientation.x = q[0]
127        pose_stamped.pose.orientation.y = q[1]
128        pose_stamped.pose.orientation.z = q[2]

```

```

125         pose_stamped.pose.orientation.w = q[3]
126
127         self.arm.set_start_state_to_current_state()
128         self.arm.set_goal_state(
129             pose_stamped_msg=pose_stamped, pose_link=self.
eef_link
130         )
131
132         plan_result = self.arm.plan()
133
134         if plan_result and plan_result.trajectory:
135             self.get_logger().info("Plan succeeded,
executing...")
136             self._display_trajectory(plan_result)
137             self.moveit.execute(plan_result.trajectory, [])
138         else:
139             self.get_logger().error("Planning failed for
this pose")
140
141         def go_through_poses(self, poses_stamped):
142             for i, pose_stamped in enumerate(poses_stamped):
143                 self.get_logger().info(f"Moving to waypoint {i
+ 1}")
144                 self.arm.set_start_state_to_current_state()
145                 self.arm.set_goal_state(
146                     pose_stamped_msg=pose_stamped, pose_link=
self.eef_link
147                 )
148                 plan_result = self.arm.plan()
149                 if plan_result and plan_result.trajectory:
150                     self._display_trajectory(plan_result)
151                     self.moveit.execute(plan_result.trajectory,
[])
152                 else:
153                     self.get_logger().error(
154                         f"Planning failed at waypoint {i + 1}"
155                     )
156                     break
157
158         def follow_cartesian_path(self, waypoints, eef_step
=0.01,
159                                   jump_threshold=0.0):
160             self._publish_path_marker(waypoints)

```

```

161
162         if eef_step <= 0:
163             sampled_waypoints = waypoints
164         else:
165             sampled_waypoints = []
166             accumulated = 0.0
167             prev = None
168
169             for wp in waypoints:
170                 if prev is not None:
171                     dx = wp.pose.position.x - prev.pose.
position.x
172                     dy = wp.pose.position.y - prev.pose.
position.y
173                     dz = wp.pose.position.z - prev.pose.
position.z
174                     accumulated += math.sqrt(
175                         dx * dx + dy * dy + dz * dz
176                     )
177
178                     if accumulated >= eef_step:
179                         sampled_waypoints.append(wp)
180                         accumulated = 0.0
181                     else:
182                         sampled_waypoints.append(wp)
183                         prev = wp
184
185             if sampled_waypoints[-1] is not waypoints[-1]:
186                 sampled_waypoints.append(waypoints[-1])
187
188             self.get_logger().info(
189                 f"Following path with {len(sampled_waypoints)}
waypoints"
190             )
191             self.go_through_poses(sampled_waypoints)
192
193     def demo_six_points(self):
194         waypoints = [
195             self._create_pose_stamped(0.4, 0.0, 0.6),
196             self._create_pose_stamped(0.4, 0.1, 0.6),
197             self._create_pose_stamped(0.4, 0.1, 0.7),
198             self._create_pose_stamped(0.3, 0.0, 0.7),
199             self._create_pose_stamped(0.3, -0.1, 0.7),

```

```

200         self._create_pose_stamped(0.3, -0.1, 0.6),
201     ]
202     self._publish_path_marker(waypoints)
203     self.go_through_poses(waypoints)
204
205     def demo_eight_shape(self):
206         waypoints = []
207         radius = 0.08
208         cx, cy, cz = 0.4, 0.0, 0.6
209
210         for angle in range(0, 628, 10):
211             t = angle / 100.0
212             x = cx + radius * math.sin(t)
213             z = cz + radius * math.sin(2 * t) * 0.5
214             waypoints.append(self._create_pose_stamped(x,
215 cy, z))
216
217         self.follow_cartesian_path(waypoints, eef_step
218 =0.005)
219
220     def demo_ellipse(self):
221         waypoints = []
222         a, b = 0.1, 0.05
223         cx, cy, cz = 0.4, 0.0, 0.6
224
225         for angle in range(0, 628, 10):
226             t = angle / 100.0
227             x = cx + a * math.cos(t)
228             z = cz + b * math.sin(t)
229             waypoints.append(self._create_pose_stamped(x,
230 cy, z))
231
232         self.follow_cartesian_path(waypoints, eef_step
233 =0.005)
234
235     def _create_pose_stamped(self, x, y, z, roll=0.0, pitch
236 =0.0,
237                             yaw=0.0):
238         pose_stamped = PoseStamped()
239         pose_stamped.header.frame_id = "panda_link0"
240         pose_stamped.pose.position.x = x
241         pose_stamped.pose.position.y = y
242         pose_stamped.pose.position.z = z

```

```
238     q = tf_transformations.quaternion_from_euler(roll,
239     pitch, yaw)
240     pose_stamped.pose.orientation.x = q[0]
241     pose_stamped.pose.orientation.y = q[1]
242     pose_stamped.pose.orientation.z = q[2]
243     pose_stamped.pose.orientation.w = q[3]
244     return pose_stamped
245
246 def main(args=None):
247     rclpy.init(args=args)
248     controller = PandaController()
249
250     controller.demo_six_points()
251     # controller.demo_eight_shape()
252     # controller.demo_ellipse()
253
254     controller.destroy_node()
255     rclpy.shutdown()
256
257
258 if __name__ == "__main__":
259     main()
```

Listing 11: panda_controller.py 完整源代码